2400-DS1AL # Algorithms for Data Science

Lecture notes

Credits: dr Krzysztof Fleszar

Lecture 9: Hard Problems

When solving an algorithmic question, we often run into the following natural question: is our algorithm optimal? We present known results of this kind, and some examples of problems which are known to be hard. We also discuss what options we have when faced with a problem that is too hard to be solved optimally.

We have already seen some "lower bound" results:

- In Lecture 5 (Sorting), we have learned that sorting cannot be done faster than in $\Omega(n \log n)$ time when we use only comparisons. However, maybe we could sort faster if our algorithm is not based on comparisons? Counting sort shows that the answer is yes for some data!
- On the first exercise sheet, we prove that finding the minimum element requires at least $n-1$ comparisons. A similar lower bound holds if we search for a value in an unsorted array. If we did it faster than in $\Omega(n)$ time, this would mean that we have not looked at the whole array, so our algorithm couldn't be correct. Thus, as long as we do not have any additional assumptions about the array (e.g., that it is sorted after all), this problem cannot be solved faster than in $\Omega(n)$ time. A similar reasoning works for many other problems.

While these results are relatively simple, it turns out to be very difficult in general to show that something is hard. Still, there is a large class of problems (called NP-complete problems) for which—though we have no proof—we have good evidence that they cannot be solved efficiently.

A Bit of Formalism

We will need a little bit of more formalism to clearly present the ideas in this lecture. Before, in the first lecture, we have defined computational problems only intuitively; now, we will define so-called *decision problems* in a more formal way.

But let's start with some intuition. Informally, a decision problem is a set of questions (inputs) for which the answers (correct outputs) are either YES or NO. For example, consider the problem of reachability in a graph: Given a graph G, can we reach vertex 2 from vertex 1? Abstractly speaking, given any input for some decision problem, the question is whether the input leads to a YES or NO answer. Formally, a decision problem can be defined as the set of all (possibly infinite many) inputs for which the answer is YES. Thus, the "problem" is to decide, whether a given input belongs to this set; hence, whether the input is an element of the decision problem (which is defined to be a set). However, the definition is not complete yet as we have still to define what we actually mean by *input*.

Let Σ be a finite set called an *alphabet*, and Σ∗ be the (infinite) set of all possible *words*. *A* word is *a finite* sequences of symbols from Σ. Think of files in a computer—all files of a computer are sequences of bytes, so Σ is the set of all possible values of a byte (0 to 255) and each file is an element of Σ∗—the set of all possible files. In theoretical computer science, we abstract from low-level technical details and have no limits on the maximum length of a file (that is, of a word); it just has to be *finite*. In particular, Σ could be any alphabet, not necessarily the set of bytes.

The input of a decision problem is a word: our algorithm takes an element of Σ∗ (e.g., it reads a file), and decides whether the answer to this "question" (i.e., the output corresponding to the input word) is YES or NO. Thus, as noted above, a decision problem can be modeled as a subset of Σ∗: the set of all input words for which our algorithm should answer YES.

For example, consider again the problem of reachability in a graph: given a graph G, can we reach vertex 2 from vertex 1? We choose some format of representing a graph in a file (e.g. a list of all edges: "1,5;1,4;4,3;3,2"). Then, the Reachability problem can be defined as the set of all encodings of graphs where 2 is reachable from 1.

Finally, we define the *size* (or *length*) of an input to be the number of its letters. In other words, we define it as the length of the character sequence of which the input consists. Usually, the input size is denoted by n.

Definition: The size (length) of an input is the length of its characters.

For example, if the input is the word "Hello", then it's size is 5 (the letter "l" is counted twice). If the the input is the number 987643210 written in decimal system, then the input size is 10. In general, a number of value v can be encoded using $O(\log v)$ bits. Or the other way around, if the input is a number of input length n (i.e. the input uses n digits), then the number's value is not larger than $2^{O(n)}$ (in standard encoding), no matter how many digits we use in our "alphabet" (binary, decimal, hexadecimal,...).

What is the purpose of this formalization?

- Decision problems (with binary answers) may look simpler than the more general problems, but actually, most of the algorithmic hardness is already present in decision problems. Thus, by restricting ourselves to decision problems, we keep the definitions simpler which helps us to better focus on the actual hard core of the problems without getting distracted by too many "unnecessary" details.
- Representing all kinds of inputs as sequences of characters ("files"/"words") allows us to precisely say what the size of a given input is, and to study complexity as a function of this size.

The Time Hierarchy Theorem

The Time Hierarchy Theorem says the following.

Time Hierarchy Theorem

If f, g are functions where g is significantly greater than f, then there exists a decision problem $L \subseteq \Sigma*$ which can be solved in $O(g)$ but not in $O(f)$.
We do not provide the precise meaning of "significantly greater" as the formulas are a bit sophisticated and may depend on the used model of computation (e.g. in Wikipedia this theorem is stated for Turing machines, not for random access machines (RAMs) that we mentioned in the first lectures). Also the two functions f and g need to fulfill a special requirement that we do not discuss here (they have to be "time-constructible"). We will just give two corollaries:

- for every k, there exists a problem which can be solved in polynomial time, but not in $O(n^k)$.
- there exists a problem which can be solved in exponential time, but not in polynomial time.

Note that the Time Hierarchy Theorem has been proven with an artificial decision problem that has been specifically constructed just for this purpose: to be solvable in $O(g)$ but not in $O(f)$ time. Thus, it is not a problem we would care about otherwise. However, it proves the existence of such problems. Consequently, there might exist also practical real world problems that have the same running time bounds.

The Complexity Class P

P (or **PTIME**) is the set ("complexity class") of all decision problems that can be solved in time that is (at most) polynomial in the length of its input. For instance, if the running time is $n2$, or $n3\log n$, or just $\log n$, all problems solvable in such times belong to P. Mathematically, if a decision problem is solvable by an algorithm with running time $nO(1)$ (note that here we use O and not Θ), then it belongs to P. Since a decision problem is itself a set

of all words ("files") for which the correct answer is YES, a complexity class as P is actually a set of sets.

To be more precise, the definition above assumes that we run our algorithm on the aforementioned Turing machine model. However, in most cases this is equivalent to assume that it runs on our RAM model: the rule is that if the running time on a RAM machine is polynomial in some parameters bounded by the input length (e.g, the number of vertices of the graph, which is shorter than the actual length of the input that has to encode the whole graph with all its vertices), then the running time on Turing machine is also polynomial (but with respect to a possibly larger polynomial).

An example of a problem in P is the Reachability problem mentioned above. It belongs to P because it can be solved in polynomial time by using breadth-first search (recall that BFS runs in linear time on our RAM model). Two other examples of problems in P are the decision variants of the Sorting problem and the Cheapest Path problem: in the first one we have to say YES or NO whether a given array of n elements is sorted. Clearly, we can check this property in polynomial time (on a RAM machine even in time linear in n) by going through the array from left to right. In the second problem, we want to know whether a given weighted graph contains a path from vertex 1 to vertex 2 of cost at most C, where C is also part of the input. This problem can be solved using Dijkstra's algorithm in polynomial time.

Usually, complexity theorists consider the class P as the class of problems that humans can actually solve in practice (using computational resources). For practical reasons, this assumption may be considered a bit doubtful. Admittedly, it is clear that, for inputs of size n=1000, a running time of $O(n3)$ is feasible, while $O(2n)$ is not. But what if we compare $O(n20)$ (allowed by P) versus $O(1.001n)$ (not allowed by P)? For relatively large values up to n<248596, the exponential function $1.001n$ is smaller, often much smaller, than the polynomial function $n20$. What about quantum computers which potentially could solve problems in polynomial time which our random access machines (RAMs) cannot? What about randomized algorithms that are not allowed by P? Some of them yield the correct answer with probability 0.999999999, which is good enough for all practical uses! However, despite these shortcomings, the simplicity of the definition of PTIME yields a very elegant theory. And the assumption that it contains problems efficiently solvable in practice is at least a good rule of thumb (as long as the exponent is a small constant).

Problems Believed to Not Be In P

Above we have seen an example of a problem belonging to P. What about problems not belonging to P? Here we present some problems which are *believed* to not be in P:

- **Knapsack**: given a knapsack with capacity M, and a target value V, and n items, where item i has weight vi and value pi, is it possible to fill our knapsack with some of these items (not exceeding its capacity) in such a way that the total value of these items is at least V?

  The Knapsack problem has been already covered in the lecture about dynamic programming. We have shown an algorithm with running in

time $\Theta(Mn)$. Though this algorithm was for the optimization variant of the problem ("what is the highest value we can achieve?"), it can of course also be used to solve the decision variant ("Can we achieve a value of at least V?"). So, does this algorithm imply that the Knapsack problem is in P given the running time of $O(Mn)$? Surprisingly, the answer is no! The value of M can be exponentially larger than the size of the input (that also encodes M). To get a feeling for it, recall that the input is a finite sequence of characters (a word), and the size of the input is the length of the sequence. In real world, the input could be a file and its size could be the number of bytes. Now, our "file representation" contains all the numbers describing the problem (item weights and values as well as the capacity M). It could be that M alone occupies a significant part of the file, say, 10% of the file contains just the digits of M. Since a number is basically always exponentially larger then the number of digits needed to encode it, we have that M is exponentially larger than than 10% times the input length. Since 10% is a only a constant, we get the running time of our dynamic programming algorithm, $\Theta(Mn)$, is at least exponential in the input length. Note that our discussion does not imply that the Knapsack problem is not in P. Maybe there is a real polynomial time algorithm for this problem. But so far, nobody has discovered it. Maybe you will? (Or show that it's not possible as it is commonly believed?)

- **Hamiltonian Cycle**: given a graph G, does there exist a cycle which goes through every vertex exactly once? (Such a path is called a "Hamiltonian cycle".)

- **Traveling Salesman Problem (TSP)**: given a weighted graph G and a number C, where every edge has a traveling cost, does there exist a tour (i.e. a cycle in which vertices can repeat) which visits all the vertices (i.e., visits every vertex at least once), and whose total cost does not exceed C? One can find a lot of applications for this problem, and yet, it is hard to solve.

- **Boolean Satisfiability (SAT)**: given a Boolean formula (e.g. `(a or b) and (not a or not b) and (a or not b)`), does there exist an assignment of true/false values to all the variables such that the formula is true? (The formula above is true when we assign true to a and false to b.) The importance of this problem will be explained later.

- **Minesweeper**: a toy problem based on the well-known Minesweeper puzzle. We are given a rectangular board with some numbers, each number gives the number of mines in adjacent cells; the positions of mines themselves are not given. Is the given board consistent, i.e., does there exist an arrangement of mines which is consistent with the given numbers?

- **Factorization**: given two numbers *n* and *a*, does *n* have a factor greater than *a*? The idea of this problem is that we want to find the prime decomposition of n, for example, 1001 = 7*11*13; this problem can be reduced to the decision problem by asking about factors greater than some *a*. This example may not be natural for data scientists, but it is important for two reasons: (a) the hardness of this problem has practical application—cryptography algorithms (e.g., used for Internet banking to authenticate bank transactions and to prevent other people from reading the customer's communication with the bank) are often based on the hardness of problems similar to this; (b) we cannot simply

try all the possible factors—our number *n* can be hundreds or thousands of digits long (and in cryptographic applications it is!) and the number of potential factors to be tested is exponential in the number of digits (hence, if we use 1000 bits, then there are about 21000 possible factors).

Reductions

For each of the problems above, we do not know whether it really cannot be solved in P. At first glance, it may appear that these open problems are independent—one could imagine that we could solve, say, SAT quickly, but not TSP. However, this is not the case.

Suppose we could solve the Traveling Salesman problem in polynomial time. Then we could also solve the Hamiltonian Cycle problem in polynomial time. Why? Well, the algorithm works as follows. For the Hamiltonian Cycle problem, we are given a graph G with n vertices. First, we assign to each edge a traveling cost of 1 and then run our assumed algorithm to Traveling Salesman problem to answer the following question: can we visit all the vertices such that the total cost of the tour is at most n? If yes, then the tour must be a Hamiltonian cycle (the edge weights of 1 and the cost bound of n prohibits that any vertex is visited twice); if not, then no Hamiltonian cycle exists. Thus, by solving the Traveling Salesman problem in polynomial time, we have also solved the Hamiltonian Cycle problem in polynomial time!

This process is a common approach in different fields as mathematics, computer science and data science—when we approach a new problem, we try to reduce it to a problem that we already know how to solve (there are even some jokes about that). Here, we are using a specific kind of reduction.

We say, there is a **polynomial time (many-one) reduction** from a problem $L1$ to a problem $L2$ if there exists an algorithm running in polynomial time that transforms any given input $w1$ of $L1$ to an input $w2$ of $L2$ in such a way that $w2$ has the same answer as $w1$, that is, the answer to the question $w1 \in L1$? should be the same as to the question $w2 \in L2$?! We say that $L1$ is reducible to $L2$ (notation: $L1 \leq L2$) if such a reduction exists.

A formal definition is as follows *(but out of scope of our course, so you can skip it)*: A polynomial time (many-one) reduction from a problem $L1$ to a problem $L2$ is a function $f: \Sigma* \to \Sigma*$, computable in polynomial time, such that $w \in L1$ if and only if $f(w) \in L2$ (i.e., for every input w, the answer for the input w for the problem $L1$ is YES if and only if the answer for the input $f(w)$ for the problem $L2$ is YES).

Above, we have shown that the Hamiltonian Cycle problem can be reduced to TSP: the reduction transforms the input graph by simply adding the weight 1 to every edge. In general, reductions can be more sophisticated, but they still have to run in polynomial time. Note that if $L1 \leq L2$ and $L2$ can be solved in polynomial time, then so can $L1$.

We can also reduce:

- Any of the problems above to the Boolean Satisfiability (SAT) problem. The practical usefulness of the SAT problem comes from the fact than many problems can be easily reduced to it.
- SAT to the Minesweeper problem. So, given a boolean formula ϕ, we can (in polynomial time) construct a Minesweeper board which is consistent if and only if ϕ is satisfiable.
- SAT to the Hamiltonian Cycle problem.
- SAT to the Knapsack problem.

Note that if $L1 \leq L2$ and $L2 \leq L3$, then $L1 \leq L3$. Therefore, any of the problems above can be reduced not only to SAT, but also to Minesweeper or to Knapsack. If we manage to solve any of these problems (SAT, TSP, Knapsack, Minesweeper, Hamiltonian Cycle) in polynomial time, then we know that all of them can be solved in polynomial time!

It is an open question whether any of these problems can be reduced to the Factorization problem.

The Complexity Class NP

All the decision problems above can be formulated in the following form: for a given input x, does there exist a "solution" y (for instance, a cycle visiting each vertex exactly once in the Hamiltonian Cycle problem, or a variable assignment that makes the given formula true in the SAT problem)? While we do not know any fast algorithm to tell us whether a solution exists, in each case we can easily tell whether a solution is correct. For instance, we can check very fast whether a cycle visits each vertex exactly once.

The complexity class **NP** is the set of all decision problems for which we can check in polynomial time whether a solution is correct.

Intuitively, a problem belongs to NP if we can answer in polynomial time any questions of the type "Is y a solution to x"? The answer is YES if only if x itself is a YES instance to our NP problem and y a corresponding solution. If the answer is NO, we basically do not know anything: it could be that x is a NO instance, but it also could be that x is a YES instance but y was a wrong solution. Here, it is important to note that we define solutions only to YES instances.

Formally, a decision problem $\mathcal{S}$ is in NP, if there is decision problem $\mathcal{S}'$ belonging to P all whose YES instances are of the form (x,y) where x is a YES instance to $\mathcal{S}$ (and for every $x \in \mathcal{S}$ we have such a pair) and where y, called the solution, has length polynomially bounded by the length of x.

Note that P is a subset of NP (is contained in NP) since all the problems in P have trivial solutions (e.g., the instance itself is a "solution": in polynomial time we can solve it and thus check whether the instance is a yes-instance; we could also take the empty word as a solution or any word and just ignore it). Therefore it is wrong to assume that NP means "Non-Polynomial" as some people sometimes suggest. Instead, NP means "Non-Deterministic Polynomial": if we "non-deterministically" guess a solution, its correctness can be checked in polynomial time. (The relation of the

term to the philosophical term "non-determinism" is historical.) Furthermore, there are problems outside NP (for instance a generalization of chess to a board of size n times n). As P is a subset of NP, these problems are consequently also outside P and therefore cannot be solved in polynomial time; hence, in no sense NP could mean "non-polynomial" as there are "non-polynomial" problems outside NP.

NP-Completeness

*A decision problem $\mathcal{L}$ is **NP-hard** if and only if every NP problem can be reduced to $\mathcal{L}$. This means that such problems are "harder than", or at least "as hard as", the whole class NP.*

*A decision problem is **NP-complete** if and only if it is NP-hard and it is in NP itself. Therefore, the NP-complete problems are the hardest problems of NP.*

We said before that each of the problems mentioned above can be reduced to SAT. In fact, it can be shown that every NP problem can be reduced to SAT; thus, SAT is NP-complete. Since SAT can be reduced, e.g., to the Minesweeper problem, and Minesweeper is also in NP, the Minesweeper problem is NP-complete, too.

There are also problems that are NP-hard but not NP-complete because they lie outside of NP, that is, are even harder than the NP-complete problems. Such an example is the aforementioned generalization of the chess problem.

The P=NP Question

By the discussion above, if one manages to solve any NP-complete problem (e.g., Minesweeper) in polynomial time, this means that any problem in NP can be solved in polynomial time, and thus, P=NP. If P does not equal NP, then NP-complete problems cannot be solved in polynomial time. The factorization problem is an example of a problem which is in NP, but is neither known to be NP-complete nor known to be solvable in polynomial time. Thus, one day it could possibly turn out that factorizing numbers can be done in polynomial time while solving Minesweeper can not. The P = NP problem is the most important open problem in computer science with a million dollar prize for solving it (although due to its practical importance in algorithmics and cryptography, it should be actually worth way more than 1000000$).

The Whole Picture

On the slides, you find a picture showing the relation of the complexity classes to each other and some example problems that they contain. Most notable, there are decision problems that are *not* solvable by any computer. For instance, the Halting problem asks the simple question of whether a given program terminates ("halts"). Though for specific input programs we might be able to check whether they terminate, we cannot come up with

a general approach that is able to check any input program. In fact, it can be mathematically proven that this problem cannot be solved by any algorithm no matter what running time or memory space we allow.

Solving Hard Problems

So, what to do when you are faced with a NP-hard problem? On the slides, you find some answers. In short:

- You can either try to solve it exactly with a brute force algorithm and hope that for your particular input it will be still fast enough; that is, that your input is not a worst case one. This is plausible for many problems.
- Or you can try to make a tradeoff (when dealing with optimization problems) between quality of solution and running time. Instead for looking for the best solution, you might be OK with a solution that is close to the best one as long as you can efficiently compute it.

Ostatnia modyfikacja: wtorek, 2 czerwca 2026, 22:23

◄ Lecture 14 Przejdź do... Lecture 14 - recording ►